

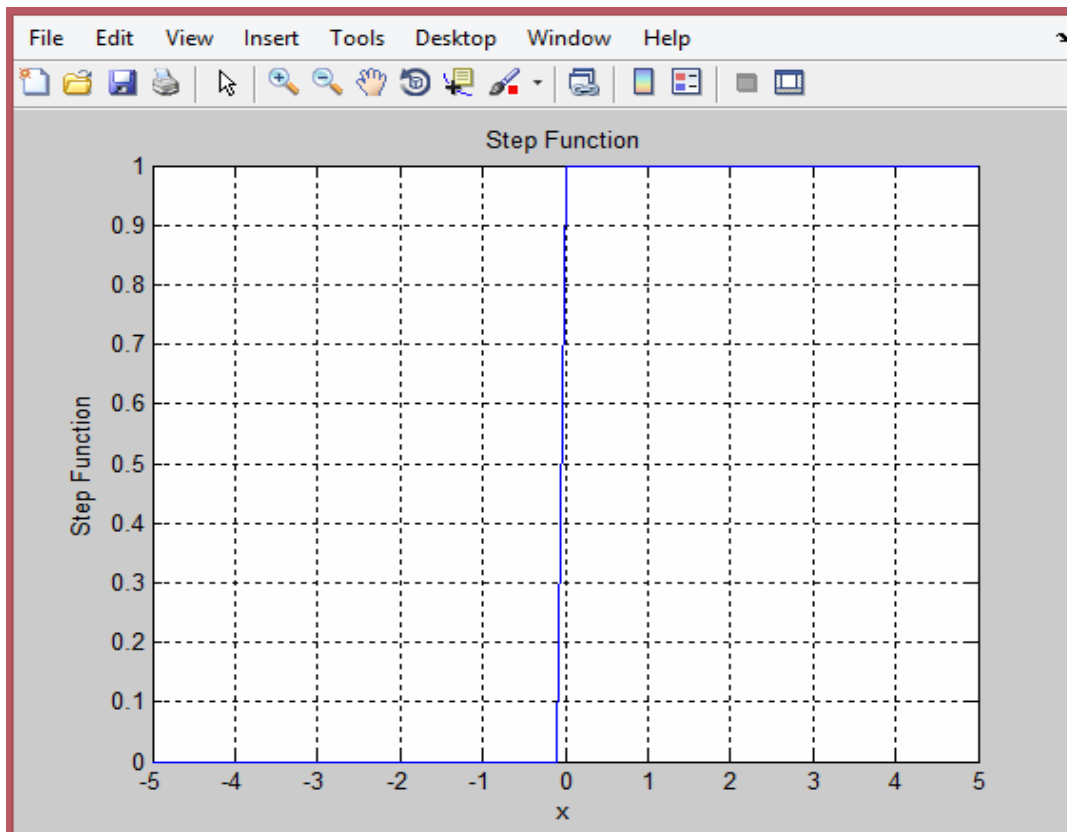
MATLAB LABORATORY PROGRAMS

1. Basic Signals and Signal Graphing: a) Unit Step, b) Rectangular, c) standard triangle d) sinusoidal and e) Exponential signal.

AIM: To write a MATLAB program to generate basics signals as Unit Step, Rectangular, Triangular, Sinusoidal, Exponential signal

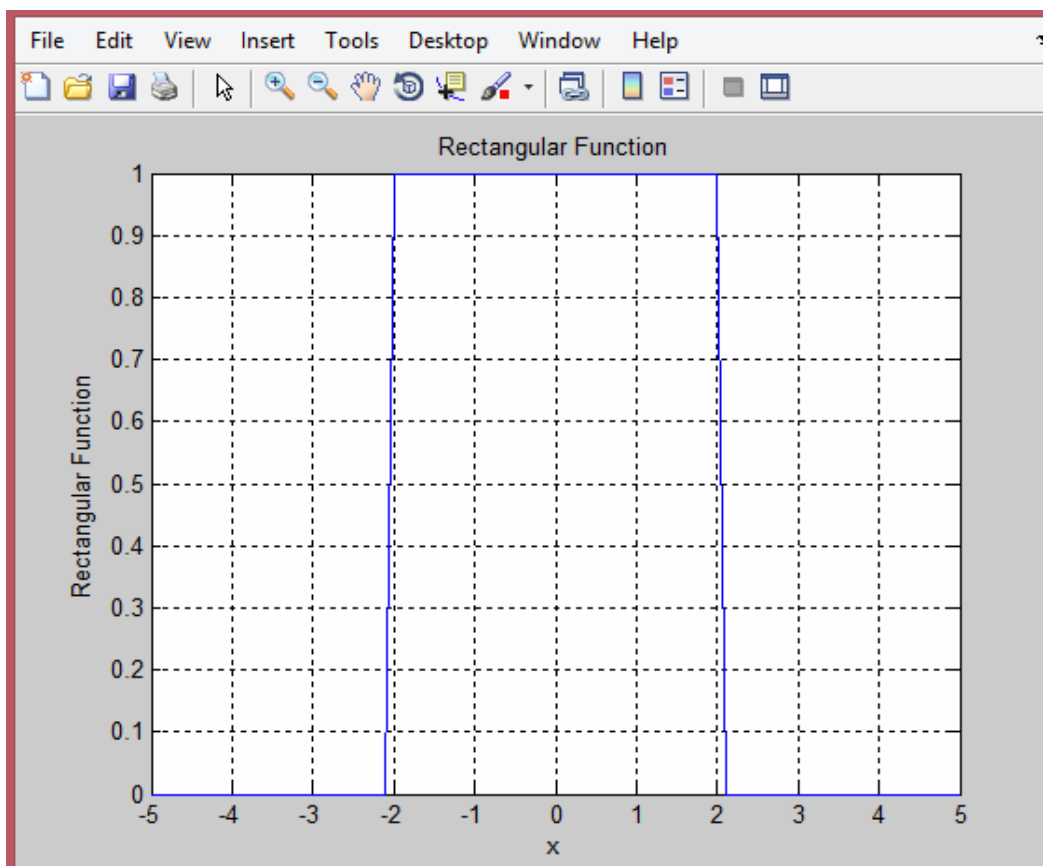
Program to generate unit step function

```
x = -5:0.1:5; % Define the range of x values
y = zeros(size(x)); % Initialize output vector
y(x >= 0) = 1; % Assign 1 to all elements where x is greater than or equal to 0
% Plot the step function
plot(x, y);
xlabel('x');
ylabel('Step Function');
title('Step Function');
grid on;
```



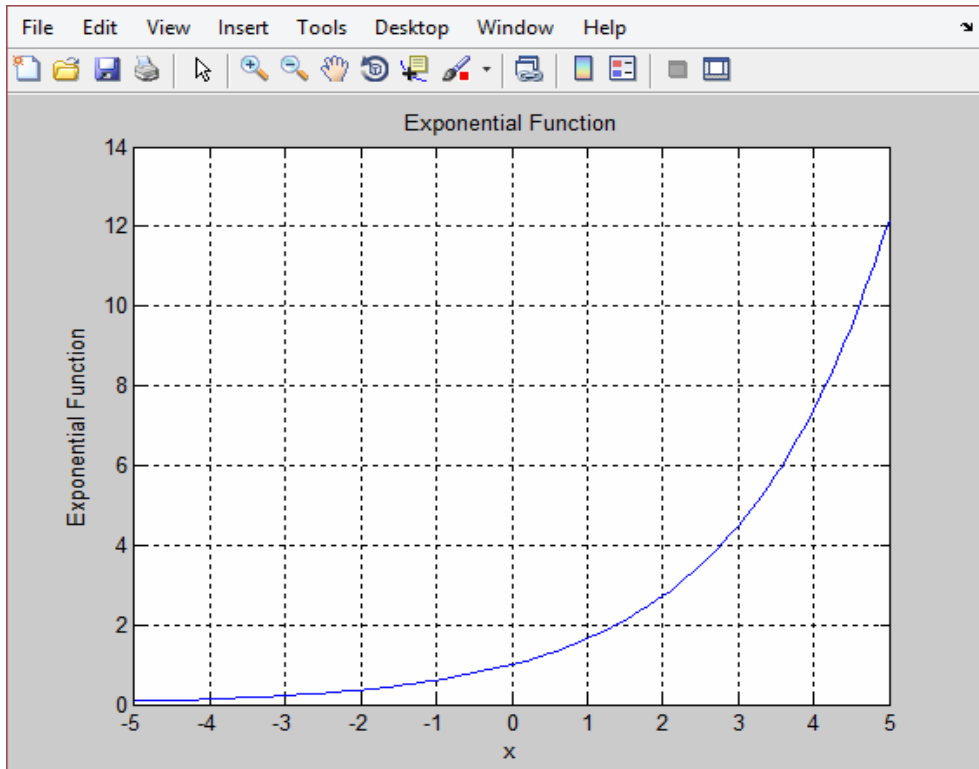
Program to generate rectangular function.

```
% x: Input vector
% a: Start point of the rectangle
% b: End point of the rectangle
x = -5:0.1:5;
a = -2;
b = 2;
y = zeros(size(x)); % Initialize output vector
y((x >= a) & (x <= b)) = 1; % Assign 1 to elements within the rectangle
% Plot the rectangular function
plot(x, y);
xlabel('x');
ylabel('Rectangular Function');
title('Rectangular Function');
grid on;
```



Program to generate exponential function.

```
% Define the range of x values
x = -5:0.1:5; % Input vector
% Define parameters for the exponential function
a = 1; % Amplitude
b = 0.5; % Exponential factor
% exponential function
y = a * exp(b * x);
% Plot the exponential function
plot(x, y);
xlabel('x');
ylabel('Exponential Function');
title('Exponential Function');
grid on;
```



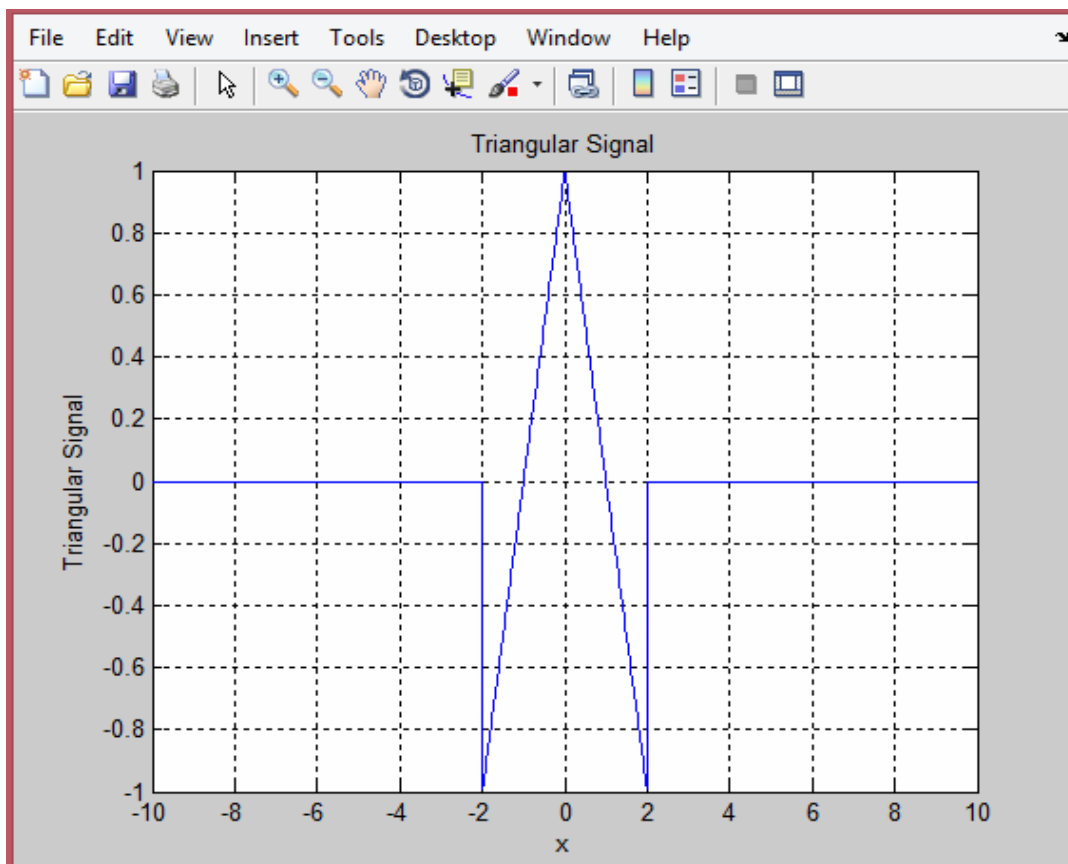
Program to generate triangular signal.

```
x = linspace(-10, 10, 1000);    % Adjust range as needed
period = 4;                     % Define the period of the triangular signal

% Normalize x to the range [-period/2, period/2)
x_normalized = mod(x + period/2, period) - period/2;

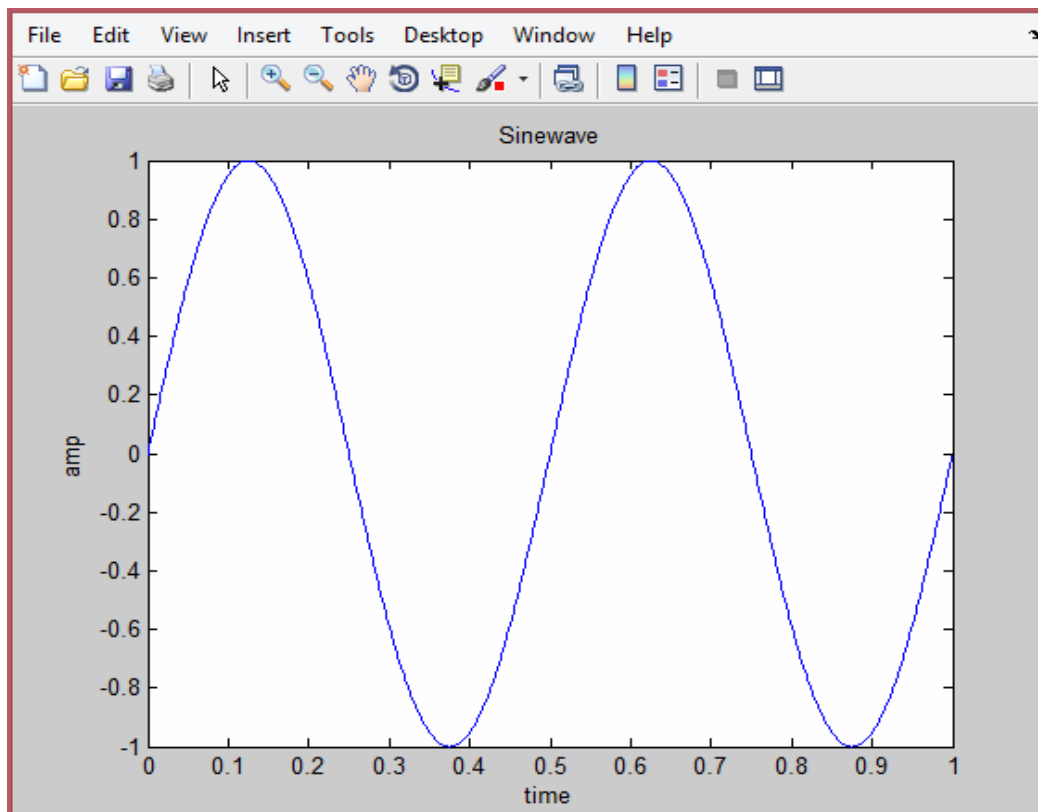
% Define the triangle function
y = 1 - 2 * abs(x_normalized) / (period/2);

% Set values outside the period range to 0
y(abs(x) >= period/2) = 0;
plot(x, y);
xlabel('x');
ylabel('Triangular Signal');
title('Triangular Signal');
grid on;
```



Program to generate sine wave

```
t=0:0.001:1;  
f=2;  
y=sin(2*pi*f*t);  
plot(t,y)  
xlabel('time');  
ylabel('Amp');  
title('Sine wave');
```



2. Illustration of signal representation in time and frequency domains for a rectangular pulse

AIM: To write a MATLAB program to generate signal representation in time and frequency domains for a rectangular pulse

% Illustration of Signal Representation in Time and Frequency Domains for a Rectangular Pulse

```
clc;  
clear all;  
close all;
```

% Parameters

```
Fs = 1000;      % Sampling frequency (Hz)  
T = 1/Fs;      % Sampling period  
duration = 1;  % Duration of signal (seconds)  
pulse_width = 0.1; % Width of rectangular pulse (seconds)  
A = 1;        % Amplitude
```

% Time vector

```
t = 0:T:duration-T;
```

% Generate rectangular pulse

```
x_rect = zeros(size(t));  
x_rect(t < pulse_width) = A;
```

% Compute Fourier Transform

```
X_rect = fft(x_rect);  
frequencies = linspace(0, Fs, length(X_rect));
```

% Plot the rectangular pulse in time domain

```
subplot(2, 1, 1);  
plot(t, x_rect, 'b', 'LineWidth', 2);  
title('Rectangular Pulse in Time Domain');  
xlabel('Time (s)');
```

```
ylabel('Amplitude');
```

```
% Plot the magnitude of the Fourier Transform in frequency domain
```

```
subplot(2, 1, 2);
```

```
stem(frequencies, abs(X_rect), 'r', 'LineWidth', 2);
```

```
title('Magnitude of Fourier Transform in Frequency Domain');
```

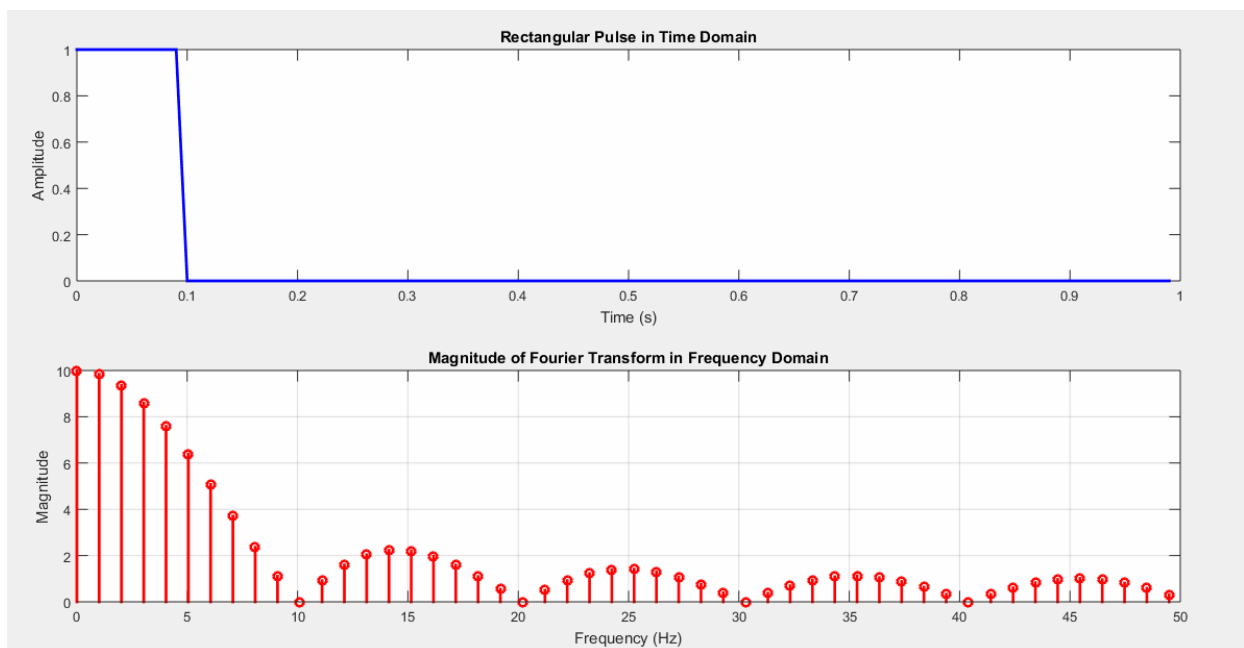
```
xlabel('Frequency (Hz)');
```

```
ylabel('Magnitude');
```

```
xlim([0, Fs/2]);
```

```
% Adjust plot
```

```
grid on;
```



3. Amplitude Modulation and demodulation: Generation and display the relevant signals and its spectrums

AIM: To write a MATLAB program to generate Amplitude Modulation and demodulation and display the relevant signals and its spectrums

```

clc;
clear all;
close all;
t=linspace(0,0.1,50000); %defining time range for the signal
fc=1000; %frequency of carrier signal
fm=500; %frequency of message signal
fs=100000;%samplingfrequency---fs>=2(fc+BW)
Am=5; %amplitude of the message signal
Ac=10; %amplitude of the carrier signal
m=Am/Ac %modulation index for the AM wave
wc=2*pi*fc*t; %carrier frequency in radians
wm=2*pi*fm*t; %message frequency in radians
ec=Ac*sin(wc); %carrier signal
em=Am*sin(wm); %message signal
y=ammod(em,fc,fs,0,Ac); %AM signal
z=amdemod(y,fc,fs,0,Ac);%demodulated AM signal
l=50000;
subplot(4,1,1),plot(t(1:l),em(1:l))
xlabel('time(sec)');
ylabel('amplitude in volts(V)');
title('MODULATING SIGNAL');
subplot(4,1,2),plot(t(1:l/2),ec(1:l/2))
xlabel('time(sec)');
ylabel('amplitude in volts(V)')
title('CARRIER SIGNAL');
subplot(4,1,3),plot(t(1:l),y(1:l))
axis([0 0.02 -20 20]) %setting axis dimensions
xlabel('time(sec)');
ylabel('amplitude in volts(V)');

```

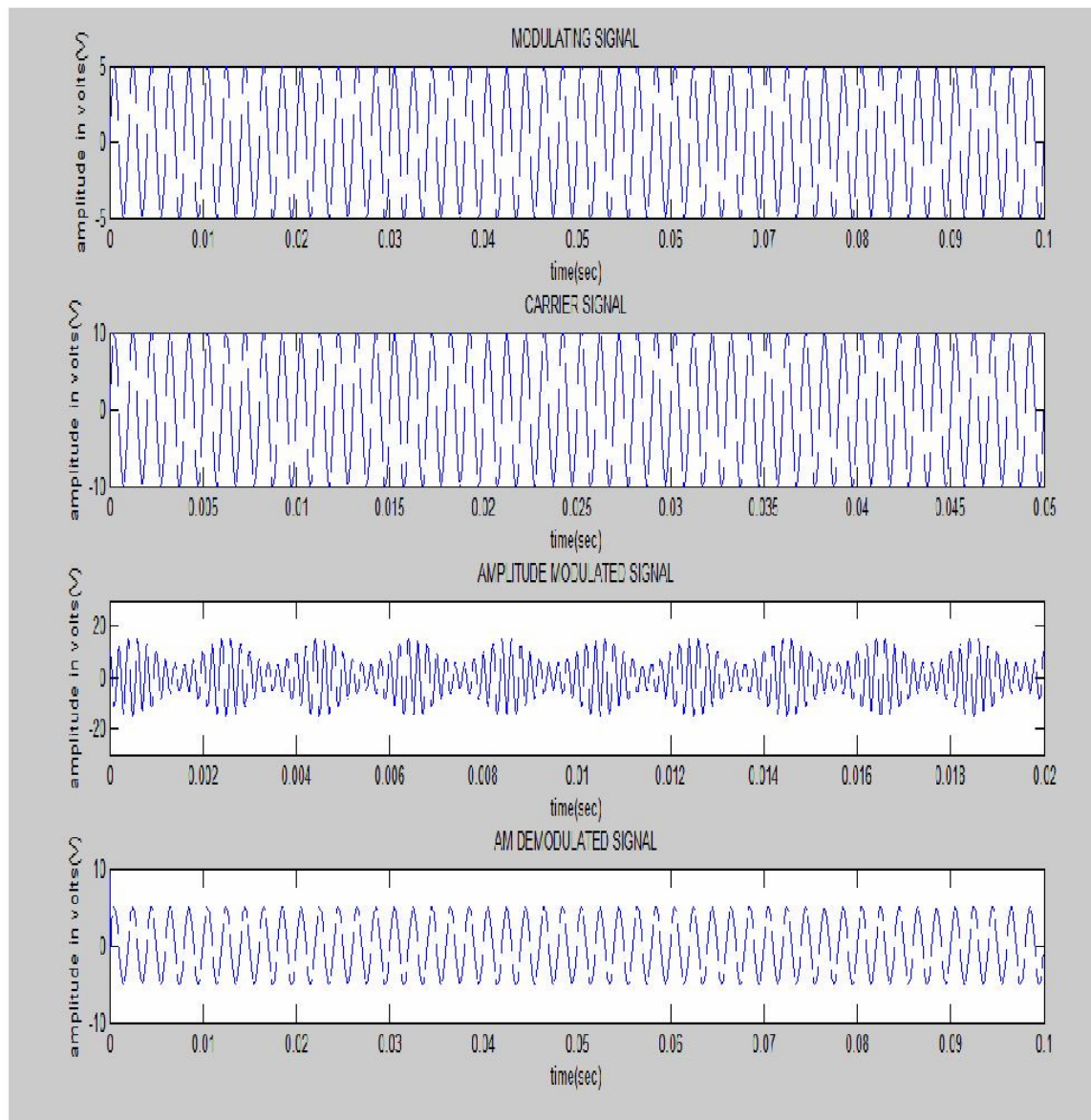
```
title('AMPLITUDE MODULATED SIGNAL');
```

```
subplot(4,1,4),plot(t(1:l),z(1:l))
```

```
xlabel('time(sec)');
```

```
ylabel('amplitude in volts(V)');
```

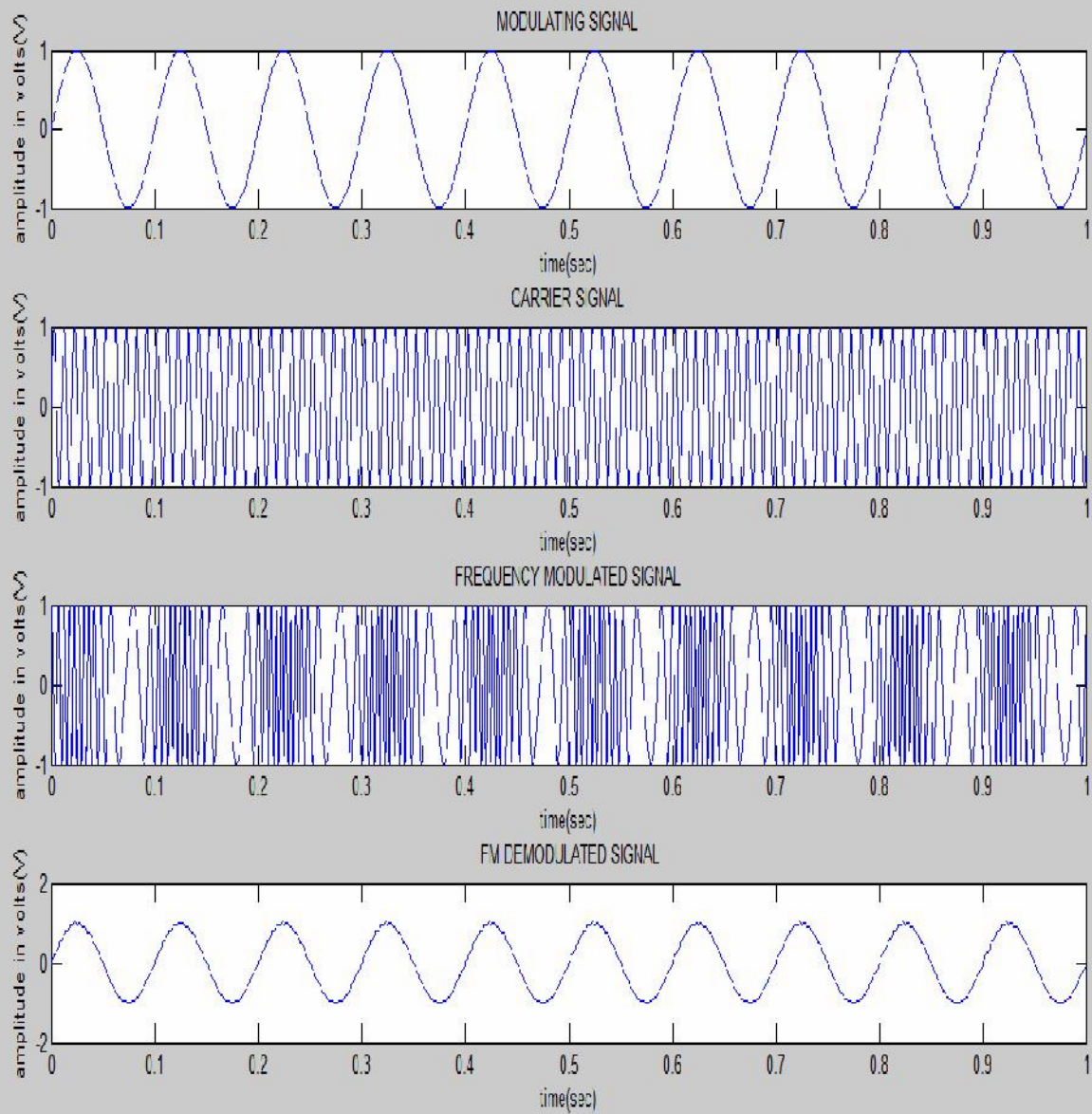
```
title(' AM DEMODULATED SIGNAL');
```



4. Frequency Modulation and demodulation: Generation and display the relevant signals and its spectrums

AIM: To write a MATLAB program to generate Frequency Modulation and demodulation and display the relevant signals and its spectrums

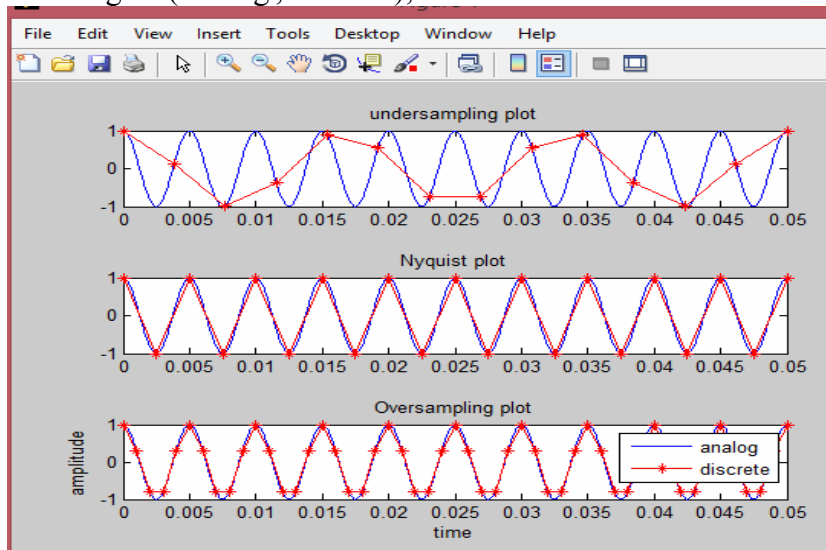
```
clc
clear all
close all
Fs = 8000; % Sampling rate of signal
Fc = 100; % Carrier frequency
t = linspace(0,1,10000); % Sampling times
x = sin(2*pi*10*t) % Channel 1
dev = 75; % Frequency deviation in modulated signal
y = fmmod(x,Fc,Fs,dev); % Modulate both channels
z = findemod(y,Fc,Fs,dev); % Demodulate both channels
subplot(411),plot(t,x)
xlabel('time(sec)');
ylabel('amplitude in volts(V)');
title('MODULATING SIGNAL');
subplot(412),plot(t,sin(2*pi*Fc*t))
xlabel('time(sec)');
ylabel('amplitude in volts(V)');
title('CARRIER SIGNAL');
subplot(413),plot(t,y)
xlabel('time(sec)');
ylabel('amplitude in volts(V)');
title('FREQUENCY MODULATED SIGNAL');
subplot(414),plot(t,z)
xlabel('time(sec)');
ylabel('amplitude in volts(V)');
title(' FM DEMODULATED SIGNAL');
```



5. Sampling and reconstruction of low pass signals. Display the signals and its spectrum

AIM: To write a MATLAB program to generate Sampling and reconstruction of low pass signals. Display the signals and its spectrum

```
tfinal=0.05;
t=0:0.00005:tfinal;
fd=input('Enter analog frequency');
%define analog signal for comparison
xt=cos(2*pi*fd*t);
%simulate condition for undersampling i.e., fs1<2*fd
fs1=1.3*fd;
%define the time vector
n1=0:1/fs1:tfinal;
%Generate the undersampled signal
xn=cos(2*pi*n1*fd);
%plot the analog & sampled signals
subplot(3,1,1);
plot(t,xt,'b',n1,xn,'r*-');
title('undersampling plot');
%condition for Nyquist plot
fs2=2*fd;
n2=0:1/fs2:tfinal;
xn=cos(2*pi*fd*n2);
subplot(3,1,2);
plot(t,xt,'b',n2,xn,'r*-');
title('Nyquist plot');
%condition for oversampling
fs3=5*fd;
n3=0:1/fs3:tfinal;
xn=cos(2*pi*fd*n3);
subplot(3,1,3);
plot(t,xt,'b',n3,xn,'r*-');
title('Oversampling plot');
xlabel('time');
ylabel('amplitude');
legend('analog','discrete');
```

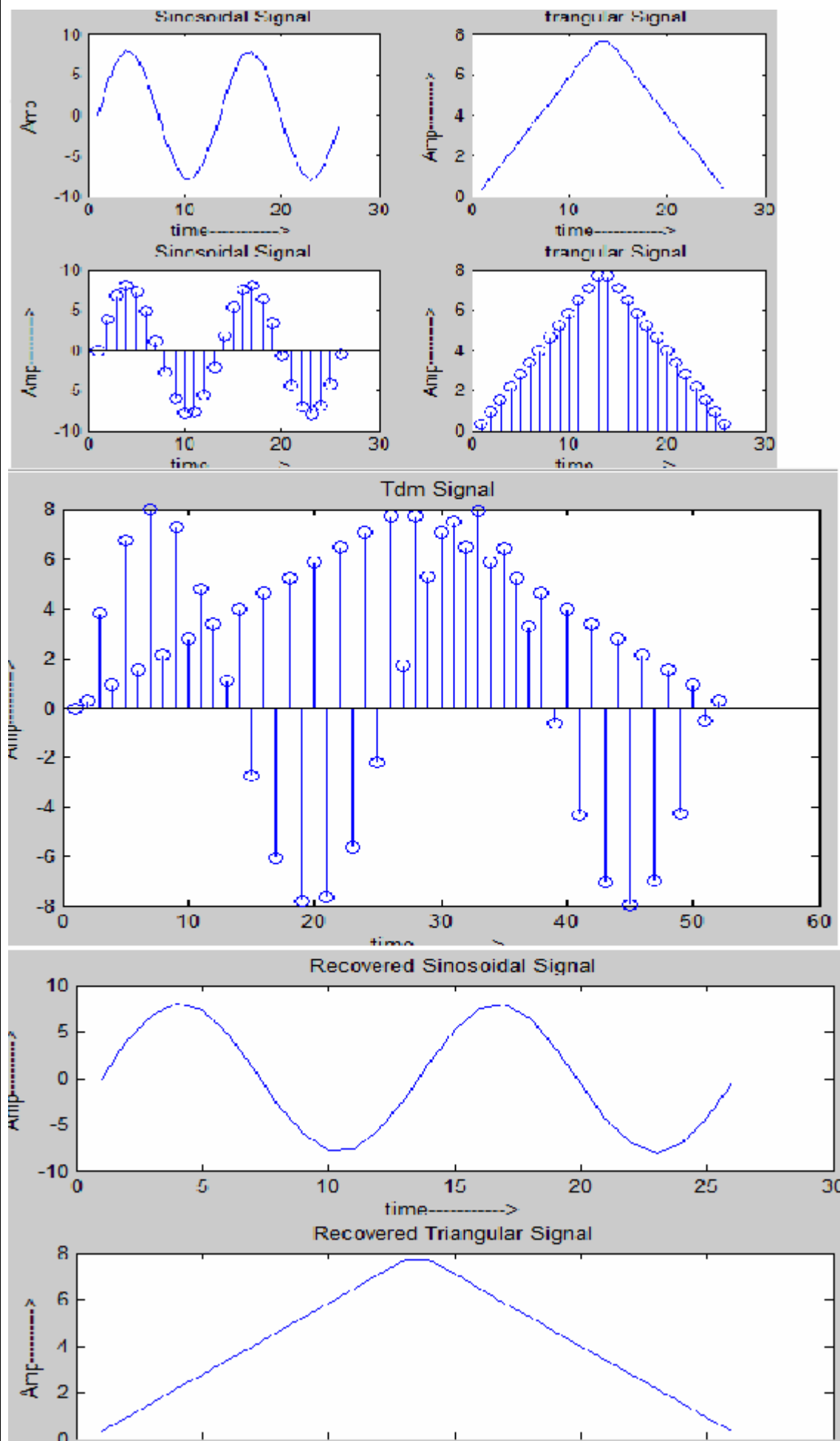


6.Time Division Multiplexing and demultiplexing

AIM: To write a MATLAB program to generate Time Division Multiplexing and demultiplexing

```
x=0:.5:4*pi;
sig1=8*sin(x);
l=length(sig1);
sig2=8*triang(l);
subplot(2,2,1)
plot(sig1);
title('Sinosoidal Signal');
ylabel('Amp');
xlabel('time----->');
subplot(2,2,2);
plot(sig2)
title('trangular Signal');
ylabel('Amp----->');
xlabel('time----->');
subplot(2,2,3)
stem(sig1);
title('Sinosoidal Signal');
ylabel('Amp----->');
xlabel('time----->');
subplot(2,2,4);
stem(sig2)
title('trangular Signal');
ylabel('Amp----->');
xlabel('time----->');
l1=length(sig1);
l2=length(sig2);
for i=1:l2
sig(1,i)=sig1(i);
sig(2,i)=sig2(i);
```

```
end
tdmsig=reshape(sig,1,2*11);
figure
stem(tdmsig);
title('Tdm Signal')
ylabel('Amp----->');
xlabel('time----->');
demux=reshape(tdmsig,2,12);
for i=1:11
sig3(i)=demux(1,i);
sig4(i)=demux(2,i);
end
figure
subplot(2,1,1)
plot(sig3)
title('Recovered Sinosoidal Signal');
ylabel('Amp----->');
xlabel('time----->');
subplot(2,1,2);
plot(sig4);
title('Recovered Triangular Signal');
ylabel('Amp----->');
```

7.PCM Illustration: Sampling, Quantization and Encoding

Aim: PCM Illustration: Sampling, Quantization and Encoding

```

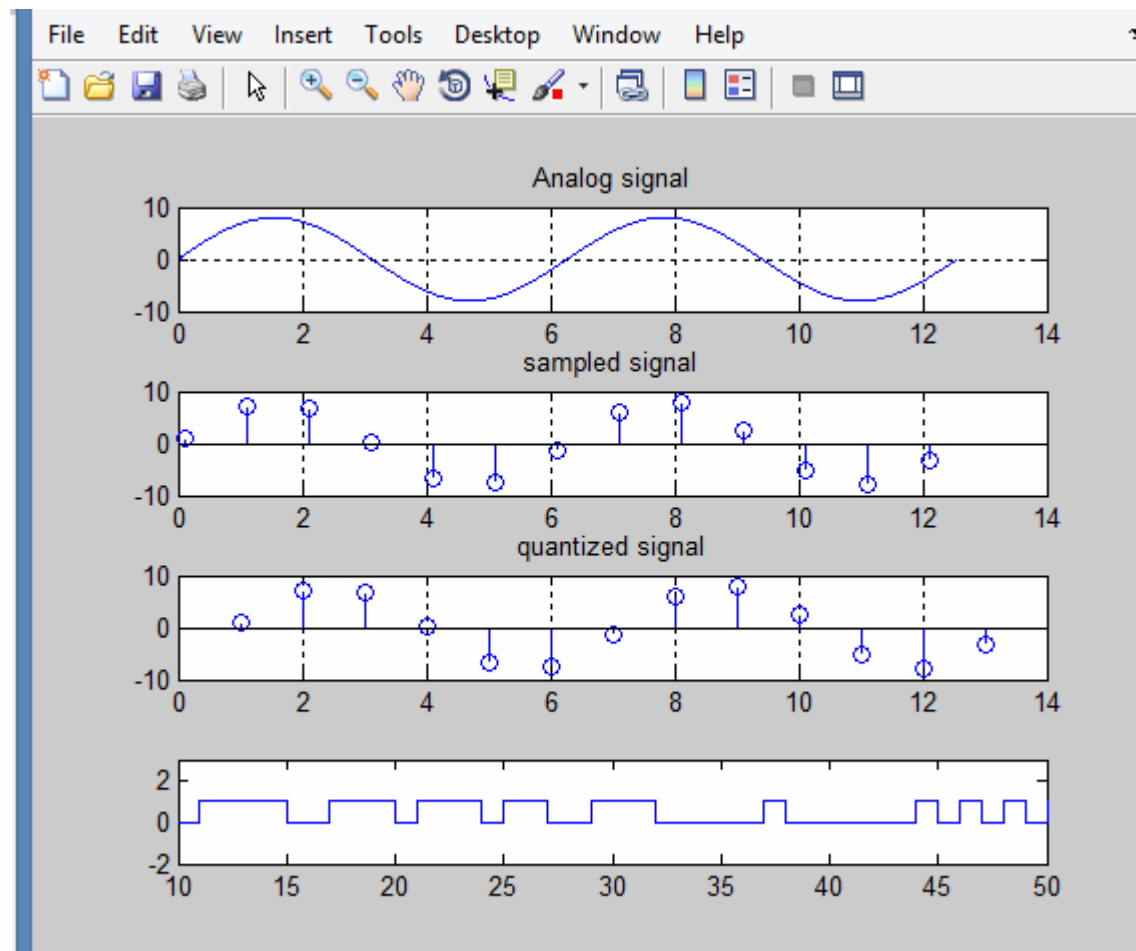
n=input('enter n value');
n1=input('enter no of sample');
x=0:1/500:4*pi;
y=8*sin(x);
subplot(4,1,1);
plot(x,y);
grid on;
title('Analog signal');
x=0.2*pi/n1:4*pi;
s=8*sin(x);
subplot(4,1,2);
stem(x,s);
grid on;
title('sampled signal');
L = 2^n;
vmax=8;
vmin=-vmax;
del=(vmax-vmin)/L;
part=vmin:del:vmax;
code=vmin-(del/2):del:vmax+(del/2);
[ind,q]=quantiz(s,part,code);
l1=length(ind);
l2=(length(q));
for(i=1:l1)
    if(ind(i)~=0)
        ind(i)=ind(i)-1;
    end
    i=i+1
end
for(i=1:l2)
    if(q(i)==vmin-(del/2))
        q(i)=vmin+(del/2);
    end
end
subplot(4,1,3);

```

```

stem(q);
grid on;
title('quantized signal');
code=de2bi(ind,'left-msb');
k=1;
for(i=1:11)
    for(j=1:n)
        coded(k)=code(i,j);
        j=j+1;
        k=k+1;
    end
    i=i+1;
end
subplot(4,1,4);
grid on;
stairs(coded);
axis([10 50 -2 3]);

```



8. Generate a) NRZ, RZ and Raised cosine pulse, b) Generate and plot eye diagram**NRZ**

```

% Define the binary sequence
binary_sequence = [1 0 1 1 0 0 1 0];

% Parameters
bit_duration = 1; % Duration of each bit
sampling_rate = 100; % Number of samples per bit duration

% Time vector for the entire sequence
t = 0:1/sampling_rate:length(binary_sequence);

% Initialize the NRZ signal
nrz_signal = zeros(1, length(t));

% Generate the NRZ signal
for i = 1:length(binary_sequence)
    if binary_sequence(i) == 1
        nrz_signal((i-1)*sampling_rate + 1:i*sampling_rate) = 1;
    else
        nrz_signal((i-1)*sampling_rate + 1:i*sampling_rate) = -1;
    end
end

% Plot the binary sequence
figure;
subplot(2,1,1);
stairs([binary_sequence binary_sequence(end)], 'LineWidth', 2);
title('Binary Sequence');
ylim([-0.5 1.5]);
xlabel('Bit Index');
ylabel('Amplitude');

% Plot the NRZ signal
subplot(2,1,2);
plot(t, nrz_signal);
title('NRZ Signal');
ylim([-1.5 1.5]);
xlabel('Time');
ylabel('Amplitude');

```

RZ

```

% Define the binary sequence
binary_sequence = [1 0 1 1 0 0 1 0];

% Parameters
bit_duration = 1; % Duration of each bit
sampling_rate = 100; % Number of samples per bit duration
half_duration = sampling_rate / 2; % Half of the sampling rate

% Time vector for the entire sequence
t = 0:1/sampling_rate:length(binary_sequence);

% Initialize the RZ signal
rz_signal = zeros(1, length(t));

```

```

% Generate the RZ signal
for i = 1:length(binary_sequence)
    if binary_sequence(i) == 1
        rz_signal((i-1)*sampling_rate + 1:(i-1)*sampling_rate + half_duration) = 1;
        rz_signal((i-1)*sampling_rate + half_duration + 1:i*sampling_rate) = 0;
    else
        rz_signal((i-1)*sampling_rate + 1:(i-1)*sampling_rate + half_duration) = -1;
        rz_signal((i-1)*sampling_rate + half_duration + 1:i*sampling_rate) = 0;
    end
end

% Plot the binary sequence
figure;
subplot(2,1,1);
stairs([binary_sequence binary_sequence(end)], 'LineWidth', 2);
title('Binary Sequence');
ylim([-0.5 1.5]);
xlabel('Bit Index');
ylabel('Amplitude');

% Plot the RZ signal
subplot(2,1,2);
plot(t, rz_signal, 'LineWidth', 2);
title('RZ Signal');
ylim([-1.5 1.5]);
xlabel('Time');
ylabel('Amplitude');

```

Raised cosine pulse

```

rolloff = 0.5
symbol_rate = 1;
span = 6;
samples_per_symbol = 100;

Fs = samples_per_symbol * symbol_rate;
Ts = 1 / symbol_rate;

t = linspace(-span/2, span/2, span * samples_per_symbol);

raised_cosine_pulse = zeros(size(t));
for i = 1:length(t)
    if abs(t(i)) == Ts / (2 * rolloff)
        raised_cosine_pulse(i) = (pi/4) * sinc(1/(2*rolloff));
    elseif abs(t(i)) < Ts / (2 * rolloff)
        raised_cosine_pulse(i) = sinc(t(i) / Ts) * cos(pi * rolloff * t(i) / Ts) / (1 - (2 * rolloff * t(i) / Ts)^2);
    else
        raised_cosine_pulse(i) = 0;
    end
end
figure;
plot(t, raised_cosine_pulse, 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Amplitude');
title(['Raised Cosine Pulse (Roll-off Factor = ', num2str(rolloff), ')']);
grid on;

```

To plot eye diagram

```
data_length = 1000;
data = randi([0, 1], 1, data_length);

% Reshape the data into a matrix with desired number of
samples per symbol
samples_per_symbol = 10;
num_symbols = data_length / samples_per_symbol;
eye_data = reshape(data, samples_per_symbol, num_symbols);

% Plot the eye diagram figure;
plot(eye_data);
title('Eye Diagram');
xlabel('Sample');
ylabel('Amplitude');
grid on;
```

9.Generate the Probability density function of Gaussian distribution function

AIM: To write a MATLAB program to generate the Probability density function of Gaussian distribution function

% Parameters

mu = 0; % Mean of the Gaussian distribution

sigma = 1; % Standard deviation of the Gaussian distribution

% Define range for x-axis

x = -5:0.1:5;

% Compute PDF

pdf = (1 / (sigma * sqrt(2 * pi))) * exp(-(x - mu).^2 / (2 * sigma^2));

% Plot PDF

figure;

plot(x, pdf, 'b', 'LineWidth', 2);

title('Probability Density Function of Gaussian Distribution');

xlabel('x');

ylabel('PDF');

grid on;